

SMART INDIA HACKATHON 2026

AI-Powered Criminal Network Analysis System

Technology stack, system architecture and phased build plan — connecting fragmented crime intelligence into an evidence-backed, explainable investigator dashboard.

PROBLEM STATEMENT

26189

DEPARTMENT

NCRB — Women Safety Division

ORGANIZATION

Ministry of Home Affairs

DOCUMENT TYPE

Architecture & Team Build Plan

AI / NLP

DATABASE

GRAPH ANALYTICS

DETECTION ENGINE

DASHBOARD

What We're Building

An investigator decision-support system. It ingests synthetic FIR text, CDR (call detail record) data and financial transaction data → extracts entities → resolves duplicate identities → builds evidence-backed relationships → stores everything in PostgreSQL → constructs a graph in NetworkX → detects suspicious patterns with explainable rules → presents it all through an interactive Streamlit + PyVis dashboard.

This document prioritises the **first working prototype (MVP)** — the smallest complete loop that proves the core idea — before listing "nice to have" additions. Every technology below is tagged by priority so the team always knows what to build first.

P1

CRITICAL

Required for the demo to work at all. Build this first — it is the entire MVP loop.

P2

SUPPORTING

Strengthens the demo (e.g. an API boundary) but the prototype survives without it.

P3

OPTIONAL / LATER

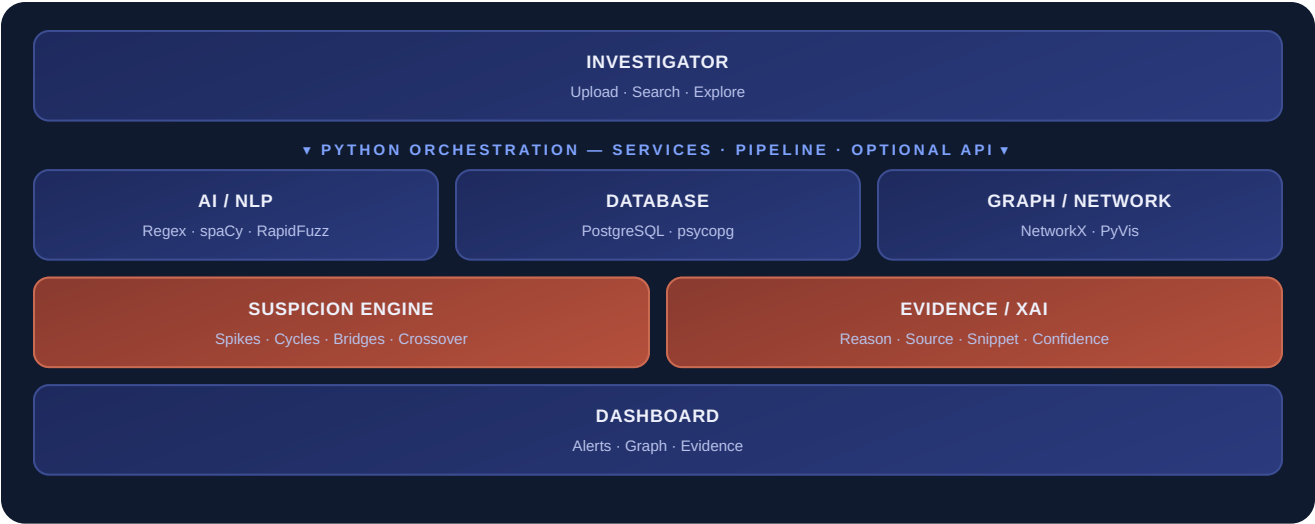
Only attempt once the P1 loop is reliable and demoable end-to-end.

The Non-Negotiable MVP Loop

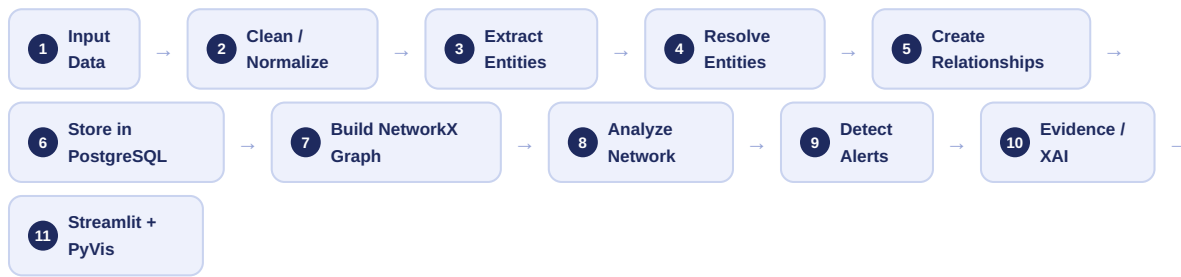


Rule: don't make every technology responsible for every problem. Use rules + graph analytics for the first suspicious-pattern engine — not a black-box classifier.

System Architecture at a Glance



End-to-End Data Flow (Implementation Sequence)



Stage	Implementation
Input	FIR text, CDR CSV, financial transactions and supported records.
Preprocess	Normalize text, phones, dates, identifiers and CSV fields.
Extract	Regex handles structured patterns; spaCy handles natural-language entities.
Resolve	Normalize → exact match → fuzzy match → attribute overlap → confidence → merge/review.
Relate	Create typed relationships such as CONTACTED, TRANSFERRED, LOCATED_AT, USED, ASSOCIATED_WITH.
Store	Persist entities, documents, transactions, relationships, alerts and evidence in PostgreSQL.
Graph	Load stored relationships into a directed/weighted NetworkX graph.
Analyze	Calculate degree/betweenness, shortest paths, connected components, community structure.
Detect	Run communication-spike, circular-flow, bridge and multi-source rules.
Explain	Attach source IDs, snippets, confidence, reason codes and relevant metrics.
Present	Streamlit filters/search/alerts + PyVis interactive network.

Part A — AI / NLP (Extraction & Resolution)

Focus on extraction and entity resolution, not a large custom neural model. Reliability, traceability and explainability matter more than raw ML sophistication for the first prototype.

Technology	Purpose	Priority	How to Implement
Regex / re	Phone, vehicle, case ID, account-like IDs, dates, emails	P1	Build and test 5–8 project-specific patterns
spaCy	PERSON, LOCATION, ORGANIZATION and text entities	P1	Pretrained NER + EntityRuler/Matcher
Pandas	Clean/process CDR and transaction CSVs	P1	read_csv, filtering, groupby, merge, missing-data handling
RapidFuzz	Duplicate/similar entity matching	P1	Normalization + similarity + supporting attributes + confidence
scikit-learn	Optional learned anomaly layer	P3	Add only after deterministic rules work
LLM	Optional evidence-grounded case summary	P3	Summarize retrieved evidence; never invent findings

4.1 — Regex Extraction (P1)

Use predictable-format extraction for phone numbers, vehicle registrations, account-like IDs, case IDs, emails and dates — more deterministic than asking an NLP model to infer structured identifiers.

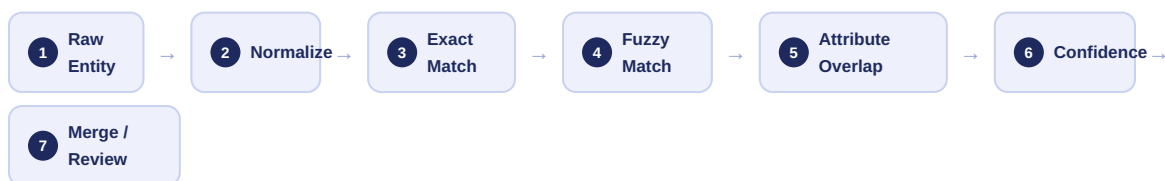
Raw: "Rahul Sharma contacted 9876543210 and used vehicle MP04AB1234. Account 4567891234 was used for the transfer."
 → PERSON: Rahul Sharma · PHONE: 9876543210 · VEHICLE: MP04AB1234 · ACCOUNT: 4567891234

4.2 — spaCy NER (P1)

Run a pretrained spaCy pipeline for PERSON, LOCATION and ORGANIZATION-like entities, then add project-specific EntityRuler/Matcher patterns. Return entity text, type, document ID and context for every extraction.

4.3 — Entity Resolution (P1)

Name variations such as "Rahul Sharma", "Rahul K Sharma" and "R. Sharma" must be evaluated as possible matches rather than treated as separate people.



Part B — Database (Persistent Source of Truth)

PostgreSQL stores canonical entities, source documents, transactions, relationships, alerts and evidence so every graph insight can be traced back to underlying records.

Table	Important Fields	Purpose
persons	id, name, normalized_name, confidence	Canonical people
phones	id, phone_number, person_id, source_id	Phone links
locations	id, name, normalized_name, latitude, longitude	Places
organizations	id, name, normalized_name	Organizations
vehicles	id, registration_number, owner_id	Vehicles / ownership

accounts	id, account_number, owner_id	Financial entities
documents	id, type, source_name, original_text, created_at	FIR / intelligence sources
transactions	id, sender_id, receiver_id, amount, timestamp, source_id	Financial flows
relationships	source_entity_id, target_entity_id, type, confidence, source_document_id, context, timestamp	Graph edges with provenance
evidence	alert_id, source_type, source_id, snippet, confidence	WHY FLAGGED support
alerts	entity_id, alert_type, severity, reason, confidence, created_at	Suspicious-pattern outputs

Design rule: every relationship record must answer — who is connected to whom, by what relationship, with what confidence, and from which source/context? PostgreSQL preserves that provenance; the same relationship records are then loaded straight into NetworkX for graph analysis.

Part C — Graph & Network Analytics

The graph layer converts database relationships into a network that exposes central actors, bridge entities, paths, groups and cycles that are difficult to see in isolated tables. Nodes = entities. Edges = evidence-backed relationships.

Node Type	Example	Typical Relationships
PERSON	Rahul Sharma	CONTACTED, TRANSFERRED, ASSOCIATED_WITH
PHONE	9876543210	USED, CONTACTED
LOCATION	Bhopal	LOCATED_AT, VISITED
ACCOUNT	Account 4567	TRANSFERRED_TO, OWNED_BY
VEHICLE	MP04AB1234	USED, OWNED_BY
ORGANIZATION	Organization X	ASSOCIATED_WITH, MENTIONED_WITH
DOCUMENT	FIR_023	MENTIONS, SOURCE_OF

Network Analytics (all P1)

Metric / Analysis	Question It Answers	Use in Prototype
Degree centrality	Who has many direct connections?	Rank highly connected entities
Betweenness centrality	Who sits between separate groups?	Bridge-entity detection
Shortest path	How are two entities connected?	Investigator tracing
Connected components	Which nodes form isolated groups?	Separate network clusters
Community detection	Which nodes form dense communities?	Group-level understanding
Cycle detection	Does flow return to its origin?	Circular money-flow detection

Suspicious-Pattern Engine (Deterministic, Explainable)

For the MVP, use deterministic rules backed by graph/data signals — never a black box. Each alert must have a reproducible condition and an attached evidence package.

Alert	Entity	Reason	Evidence
COMMUNICATION SPIKE	Rahul	42 calls vs baseline 7/day	CDR source IDs + time window
CIRCULAR MONEY FLOW	Rahul / Amit / Suresh	Directed cycle detected	Transaction IDs + amounts
BRIDGE ENTITY	Rahul	High betweenness / community bridge	Metric + paths
MULTI-SOURCE CROSSOVER	Rahul	Appears across multiple source types	Source records + snippets

Rule sketch: `if current_calls > historical_average × threshold: create_alert("COMMUNICATION_SPIKE")` — communication spikes compare volume against a historical/synthetic baseline; circular flow detects short directed cycles (e.g. Rahul → Amit → Suresh → Rahul); bridge entities use betweenness centrality; multi-source crossover finds canonical entities appearing across FIR + CDR + financial records.

Evidence & Explainability (XAI)

The system should never stop at "Person X is suspicious." It must answer **"WHY FLAGGED?"** with concrete, inspectable evidence: reason code, canonical entity, source IDs, context/snippet, confidence, graph metrics and rule details (threshold, time window, cycle or path).

WHY FLAGGED?

Entity: Rahul Sharma · **Alert:** MULTI-SOURCE CROSSOVER

Reason: Rahul is linked to the same canonical entity across FIR_023, CDR_441 and transaction records.

Evidence: FIR snippet mentions Rahul; CDR shows communication with Amit; transaction record shows a transfer to Amit.

Confidence: 0.94 · **Graph signal:** high betweenness centrality.

Design constraint: the system is decision support. An alert is a signal requiring investigator review, not a determination of guilt. The prototype uses only synthetic/fictitious data.

Part D — Frontend & Backend

A single investigator-oriented Streamlit dashboard. The MVP does not need a separate React application — keep the interface focused on upload, analysis, search, alerts, evidence and graph exploration.

Streamlit Dashboard Features

P1

Upload FIR text, CDR CSV and transaction CSV · "Analyze Case" button · Overview cards (entities, relationships, alerts, communities) · Entity search and filters · Alert list with severity and reason · WHY FLAGGED evidence panel · Interactive PyVis graph · Shortest-path exploration · Source/evidence tables.

Option	Architecture	Use When
A — Fastest MVP	Streamlit → Python services → PostgreSQL	Time is tight; recommended baseline P1
B — API architecture	Streamlit → FastAPI → Python services → PostgreSQL	Team is comfortable with APIs and wants separation P2

PyVis flow: NetworkX → PyVis → interactive HTML → Streamlit. Use node labels/types and edge relationship labels; use graph metrics to communicate structural importance.

Final Technology Stack Summary

Layer	Technology	Priority
Language	Python	P1
Data processing	Pandas	P1
NLP	spaCy	P1
Pattern extraction	Regex / re	P1
Entity resolution	RapidFuzz	P1
Database	PostgreSQL	P1
DB driver	psycopg	P1
Graph	NetworkX	P1
Visualization	PyVis	P1
Detection	Python rules	P1
Frontend	Streamlit	P1 / P2
Backend API	FastAPI	P2 / P3
ML	scikit-learn	P3
Collaboration	Git + GitHub	P1

Scope Control — What NOT to Build First

Do not overengineer the first prototype. These are explicitly out of scope until the MVP is reliable:

Neo4j as mandatory DB

React + Cytoscape frontend

BERT/transformer-heavy NLP

Graph Neural Networks

Deep anomaly detection

Multilingual NLP

Enterprise auth/authz

Microservices/Kubernetes

Cloud deployment

Black-box ML classifier

Team Division of Work

Ownership is split by module below. Every member should still understand the end-to-end pipeline well enough to debug and explain their neighbour's part in the demo.

MODULE 1

AI / ML — Extraction & Resolution

Aniruddha

- Build 5–8 Regex patterns (phone, vehicle, account, case ID, dates)
- Configure spaCy NER + EntityRuler/Matcher for PERSON/LOCATION/ORG
- Implement RapidFuzz-based entity resolution pipeline (normalize → exact → fuzzy → attribute overlap → confidence → merge)
- Pandas cleaning for CDR & transaction CSVs
- Define the extraction output contract (entity text, type, doc ID, context) for downstream modules

Stack: Python, spaCy, Regex, RapidFuzz, Pandas

MODULE 2

Database & Graph

Udit

Vanshika

- Design & migrate PostgreSQL schema (persons, phones, accounts, relationships, evidence, alerts, etc.)
- Build psycopg data-access layer and queries
- Construct the NetworkX directed/weighted graph from stored relationships
- Implement centrality, shortest-path, components & community detection
- Own the four detection rules (spike, circular flow, bridge, crossover) since they run directly on graph/DB data

Stack: PostgreSQL, psycopg, NetworkX, Pandas, RapidFuzz (resolution support)

MODULE 3

Backend / Orchestration

Aditi

Vanshika

- Wire ingestion → extraction → resolution → storage → graph → detection into one pipeline
- Build the evidence engine (attach source/context/reason/confidence/metrics to each alert)
- Handle file uploads (FIR text, CDR CSV, transaction CSV) and validation
- Optional FastAPI layer if a clean API boundary is needed (Option B architecture)
- Config, environment management, and integration testing across modules

Stack: Python (glue/orchestration), FastAPI (optional), PostgreSQL integration

MODULE 4

Frontend, UI & Presentation

Udit

Alafiya

Aditiya

- Build the Streamlit dashboard: upload, "Analyze Case", overview cards, filters
- Integrate the interactive PyVis network view + shortest-path exploration
- Design the alert list and "WHY FLAGGED" evidence panel UI
- Own visual design/UX polish for the investigator dashboard
- Prepare the hackathon PPT — architecture visuals, demo script, problem framing

Stack: Streamlit, PyVis (HTML embed), presentation design tools

Cross-Module Understanding (Debugging Rule)

For every module, the owner should know: what data enters the function, what comes out, which database table or graph structure is touched, and what condition creates an alert. AI-generated boilerplate can accelerate implementation, but the team must be able to explain and test it live during judging.

Final Demo Acceptance Checklist

- | | |
|--|---|
| ☒ Synthetic FIR text uploads successfully | ☒ Communication spike alert works |
| ☒ CDR CSV uploads successfully | ☒ Circular money-flow alert works |
| ☒ Financial transaction CSV uploads successfully | ☒ Bridge-entity alert works |
| ☒ Regex extracts structured identifiers | ☒ Multi-source crossover alert works |
| ☒ spaCy extracts text entities | ☒ Every alert has a WHY FLAGGED explanation |
| ☒ Entity resolution links/merges some duplicate records | ☒ Evidence points back to source records/snippets |
| ☒ PostgreSQL stores canonical entities and relationships | ☒ PyVis graph is interactive |
| ☒ Relationships retain source/context and confidence | ☒ Streamlit runs the complete flow |
| ☒ NetworkX graph is generated | ☒ The team can explain what each node, edge, metric and alert means |
| ☒ Degree/betweenness or another meaningful metric is shown | ☒ Demo uses synthetic/fictitious data and states decision-support scope |

Raw fragmented data → entities → resolved entities → relationships → graph → suspicious pattern → evidence → investigator dashboard.

A smaller stack that reliably demonstrates the complete evidence-backed pipeline beats a large stack of unused technologies.